CICLO FORMATIVO DE GRADO SUPERIOR DAM / DAW

Desarrollo de Interfaces (DI)

PRÁCTICA DE ESTILOS CON SASS

Preprocesamiento de CSS con Sass en Ionic & Angular

Implementación de diseño responsivo con variables, mixins y nesting modular en arquitectura Angular Standalone, análisis de compilación en caliente y contraste con el flujo manual de Sass.

ALUMNO:
Daniel Daroca

FECHA:
23 de Octubre de 2026

ENTORNO TECNOLÓGICO:
Ionic 8+ / Angular 22 / Dart Sass / TypeScript

REPOSITORIO GITHUB:
https://github.com/danidaroca/ionic-sass-actividad

1. Enunciado y Objetivos de la Actividad

El objetivo de esta práctica es dominar la integración y el flujo de trabajo de **Sass (SCSS)** dentro del ecosistema moderno de desarrollo móvil y web con **Ionic** y **Angular**.

> **Requisitos especificados en el enunciado:**
>
> 1. Crear un proyecto **Ionic - Angular** (utilizando componentes Standalone).
> 2. Trabajar sobre la página/componente ya generada llamada **home**.
> 3. Permitir el uso de etiquetas HTML estándar y componentes de Ionic: **título + botón + imagen**.
> 4. **Título:** en rojo, transformado a mayúsculas y centrado.
> 5. **Botón:** alineado a la derecha y estilizado en color azul.
> 6. **Imagen:** centrada en la pantalla con borde en color verde de grosor **3px**.
> 7. Aplicar todo el diseño mediante hojas de estilo **Sass**.
> 8. Responder de forma argumentada a las preguntas técnicas planteadas:
>   - *¿El componente usa Sass por defecto?*
>   - *¿Aplico Sass pero se compila solo? ¿Por qué con `ionic serve` no se ejecuta `npm run watch:sass` ?*

2. Creación del Proyecto Ionic - Angular

Para garantizar el uso de las versiones más estables y modernas, se ha generado el proyecto mediante Ionic CLI especificando la tipología `angular-standalone` y la plantilla `blank`:

```
ionic start Sass blank --type=angular-standalone --no-interactive
```

Esta instrucción prepara un proyecto Angular completo con Capacitor, configurando los esquemas automáticos para componentes independientes (Standalone) y estilos con extensión `.scss`.

```
Sass/ ├── src/ │ ├── app/ │ │ ├── home/ │ │ │ ├── home.page.html # Vista con componentes
HTML e Ionic │ │ │ ├── home.page.scss # Hoja de estilos SCSS modular del componente │ │ │
├── home.page.ts # Componente Angular Standalone │ ├── assets/ │ │ └── sass-banner.svg #
Gráfico ilustrativo de Sass & Ionic │ ├── theme/ │ │ ├── abstracts/ │ │ │ ├──
_variables.scss # Tokens y variables Sass (colores, bordes) │ │ │ └── _mixins.scss # Mixins
reutilizables (flexbox, marcos) │ │ └── variables.scss # Paleta de Ionic Framework ├──
angular.json # Definición del pipeline de compilación └── package.json # Dependencias y
scripts de ejecución
```

3. Implementación del Diseño con Sass Modular

Siguiendo la arquitectura profesional recomendada en el *Patrón 7-1* (visto en las sesiones teóricas previas `patron-7-1-demo` y `m9-3-sass-fundamentals`), se ha estructurado una capa de abstracciones dentro de `src/theme/abstracts/` que contiene las variables y los mixins del sistema de diseño, importados mediante la directiva moderna `@use` de Dart Sass.

3.1. Definición de Tokens y Variables (`_variables.scss`)

Se centralizan todos los valores requeridos en el enunciado para asegurar un código mantenible y desacoplado:

```
// src/theme/abstracts/_variables.scss

// Requisito 1: Título en rojo, mayúsculas y centrado
$title-color: #d32f2f;
$title-transform: uppercase;
$title-align: center;

// Requisito 2: Botón en azul alineado a la derecha
$button-bg: #1976d2;
$button-bg-hover: #1565c0;
$button-text-color: #ffffff;
$button-radius: 8px;

// Requisito 3: Imagen con borde verde de 3px
$img-border-width: 3px;
$img-border-style: solid;
$img-border-color: #2e7d32;
$img-border-radius: 12px;
```

3.2. Mixins de Maquetación y Bordes (`_mixins.scss`)

```
// src/theme/abstracts/_mixins.scss
@use 'variables' as v;

@mixin flex-center {
  display: flex;
  justify-content: center;
  align-items: center;
}

@mixin flex-align-right {
  display: flex;
  justify-content: flex-end;
  align-items: center;
}
```

```
@mixin framed-border($width: v.$img-border-width, $style: v.$img-border-style, $color: v.$im
  border: $width $style $color;
  border-radius: $radius;
  box-shadow: 0 6px 18px rgba(46, 125, 50, 0.2);
}

@mixin styled-button($bg: v.$button-bg, $hover-bg: v.$button-bg-hover, $text: v.$button-text
  background-color: $bg;
  color: $text;
  border: none;
  border-radius: v.$button-radius;
  padding: 12px 24px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.25s ease-in-out;
  &:hover {
    background-color: $hover-bg;
    transform: translateY(-2px);
  }
}
```

3.3. Estilos del Componente Home (`home.page.scss`)

En `home.page.scss` se aplican los tres requerimientos usando anidamiento jerárquico (*nesting*), llamadas a mixins con `@include` y selectores contextuales:

```
@use '../../theme/abstracts/variables' as v;
@use '../../theme/abstracts/mixins' as m;

.page-container {
  // Requisito 1: Título en rojo, mayúsculas y centrado
  .assignment-header .assignment-title {
    color: v.$title-color;
    text-transform: v.$title-transform;
    text-align: v.$title-align;
    font-size: 2rem;
    font-weight: 800;
  }

  // Requisito 2: Botón a la derecha en azul
  .button-section {
    @include m.flex-align-right;
    .btn-sass-html {
      @include m.styled-button(v.$button-bg, v.$button-bg-hover, v.$button-text-color);
    }
    .btn-sass-ionic {
      --background: #{v.$button-bg};
      --color: #{v.$button-text-color};
    }
  }

  // Requisito 3: Imagen centrada con borde verde de 3px
```

```
  .image-section .image-center-container {
    @include m.flex-center;
    .framed-banner-img {
      display: block;
      margin: 0 auto;
      @include m.framed-border;
    }
  }
}
```

4. Respuestas Argumentadas a las Preguntas Planteadas

Pregunta 1: ¿El componente usa Sass por defecto?

Respuesta directa: SÍ, Ionic con Angular utiliza Sass (SCSS) por defecto en todos sus componentes y páginas.

Fundamentación técnica:

1. **Configuración en `angular.json`:** Al generar un proyecto Ionic Angular, el archivo de configuración global define explícitamente que los generadores de código (*schematics*) creen componentes y páginas con extensión `scss`:

   ```
   "schematics": {
     "@ionic/angular-toolkit:page": {
       "styleext": "scss",
       "standalone": true
     }
   },
   "inlineStyleLanguage": "scss",
   "styles": ["src/global.scss", "src/theme/variables.scss"]
   ```

2. **Enlace en el decorador `@Component`:** El componente inicial `src/app/home/home.page.ts` declara:

   ```
   @Component({
     selector: 'app-home',
     templateUrl: 'home.page.html',
     styleUrls: ['home.page.scss']
   })
   ```

   Ionic adopta de forma predeterminada la sintaxis `.scss` (Sassy CSS) frente al CSS clásico o Sass indentado antiguo (`.sass`).

3. **Ecosistema global de temas:** Toda la arquitectura de theming de Ionic Framework (`src/theme/variables.scss` y `src/global.scss`) está diseñada nativamente en Sass, permitiendo sobreescribir CSS Custom Properties y variables SCSS de manera unificada.

Pregunta 2: ¿Aplico Sass pero se compila solo? ¿Por qué no hacemos `npm run watch:sass` ?

Respuesta directa: SÍ, Sass se compila automáticamente y en memoria al ejecutar `ionic serve` . No es necesario ni aplicable ejecutar `npm run watch:sass` .

Fundamentación técnica y comparativa de flujos de trabajo:

Aspecto	Proyectos Web Planos (p. ej. `m9-3-sass-fundamentals`)	Proyectos Ionic - Angular (`ionic serve`)
Pipeline de construcción	Inexistente. No hay empaquetador (Webpack, Vite o esbuild).	Integrado mediante `@angular/build:application` (con motor Vite + esbuild).
Compilador de Sass	Debe instalarse manualmente en `devDependencies` (paquete `sass`).	Viene integrado y gestionado internamente por el Angular CLI.
Modo de escucha (Watch)	Requiere un script en `package.json`: `"watch:sass": "sass --watch scss:dist/css"`.	Automático e inherente al comando `ionic serve` (que delega en `ng serve`).
Destino de la compilación	Escribe archivos físicos `.css` y `.css.map` en la carpeta `dist/css` en disco.	Compilación en memoria virtual (Virtual File System / RAM), optimizando I/O de disco.
Recarga de estilos	Requiere recargar manualmente el navegador o usar extensiones como Live Server.	Aplica **Hot Module Replacement (HMR)** instantáneo; los estilos se actualizan al guardar sin perder el estado de la aplicación.

> **Conclusión técnica:** En Ionic con Angular, el desarrollador solo se enfoca en escribir su código en archivos `.scss`. El proceso de compilación, resolución de rutas, vendor-prefixing, optimización y refresco en el navegador corre de forma transparente y automática dentro del ciclo de vida de `ionic serve`.

5. Código Fuente Completo y Comentado

5.1. Plantilla Vista (`src/app/home/home.page.html`)

```
<ion-header [translucent]="true">
  <ion-toolbar color="primary">
    <ion-title>Actividad Sass en Ionic & Angular</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content [fullscreen]="true" class="ion-padding page-container">
  <div class="content-wrapper">

    <!-- REQUISITO 1: TÍTULO EN ROJO, MAYÚSCULAS Y CENTRADO -->
    <header class="assignment-header">
      <h1 class="assignment-title">DISEÑO Y ESTILOS CON SASS</h1>
      <p class="subtitle">Práctica de Desarrollo de Interfaces (DI) - DAM2</p>
    </header>

    <!-- REQUISITO 2: BOTÓN A LA DERECHA EN AZUL -->
    <div class="button-section">
      <span class="button-label">Alineación a la derecha:</span>
      <div class="buttons-group">
        <button class="btn-sass-html" (click)="onButtonClick('Botón HTML estándar')">
          Botón HTML (Azul)
        </button>
        <ion-button class="btn-sass-ionic" (click)="onButtonClick('Componente Ionic')">
          Botón Ionic (Azul)
        </ion-button>
      </div>
    </div>

    <!-- Feedback dinámico al interactuar con el botón -->
    @if (buttonClickMessage) {
      <div class="feedback-alert">
        {{ buttonClickMessage }}
      </div>
    }

    <!-- REQUISITO 3: IMAGEN CENTRADA CON BORDE VERDE DE GROSOR 3PX -->
    <section class="image-section">
      <h3 class="section-title">Imagen con Borde Verde (3px)</h3>
      <div class="image-center-container">
        <img
          src="assets/sass-banner.svg"
          alt="Banner explicativo Sass con arquitectura modular"
          class="framed-banner-img"
        />
      </div>
      <p class="image-caption">
        Borde aplicado mediante mixin Sass: <code>3px solid #2e7d32</code>
```

```
      </p>
    </section>

  </div>
</ion-content>
```

5.2. Componente TypeScript (`src/app/home/home.page.ts`)

```
import { Component } from '@angular/core';
import {
  IonHeader, IonToolbar, IonTitle, IonContent,
  IonButton, IonCard, IonCardHeader, IonCardTitle,
  IonCardSubtitle, IonCardContent
} from '@ionic/angular';

@Component({
  selector: 'app-home',
  templateUrl: 'home.page.html',
  styleUrls: ['home.page.scss'],
  imports: [
    IonHeader, IonToolbar, IonTitle, IonContent,
    IonButton, IonCard, IonCardHeader, IonCardTitle,
    IonCardSubtitle, IonCardContent
  ],
})
export class HomePage {
  buttonClickMessage = '';

  onButtonClick(source: string) {
    this.buttonClickMessage = `¡Acción ejecutada desde el botón (${source}) con estilo azul
  }
}
```

6. Verificación de Compilación y Puesta en Marcha

El proyecto ha sido compilado exitosamente para producción utilizando el compilador oficial de Angular y Vite:

```
$ npm run build

> Sass@0.0.1 build
> ng build

√ Building ...
Application bundle generation complete. [10.323 seconds]
Output location: C:\Users\Daniel\Desktop\DI\Sass\www
```

> **Estado de compilación:** Cero errores, cero advertencias críticas. Todas las hojas de estilo Sass compilaron correctamente y los componentes standalone quedaron empaquetados en la carpeta `www/`.

7. Enlace de Entrega en GitHub

El código fuente íntegro de la práctica está versionado en Git y preparado para su clonación y revisión:

> **Repositorio GitHub:**
> **https://github.com/danidaroca/ionic-sass-actividad**
>
> **Comandos para clonar y ejecutar en local:**
>
> ```
> git clone https://github.com/danidaroca/ionic-sass-actividad.git
> cd ionic-sass-actividad
> npm install
> ionic serve
> ```